INDIAN ARMY INTERNSHIP PROGRAMME (IAIP) : 2026

HQ Southern Command — Technical Assessment Report

Domain: Process Automation & App Development
Candidate: Saket Rajak
GitHub: Saketrajak/Nxt-Gen-Employee-Management-System
Assessment Score: Out of 100 Marks

1. ENGINEERING ASSUMPTIONS (MANDATORY DECLARATION)

In accordance with the assessment instructions permitting reasonable assumptions, the following foundational architectural assumptions were established at the inception of the solution:

1. **Employee Code Generation & Immutability:** The alphanumeric Employee Code (e.g., `I0001`, `H0001`) reflects an employee's active departmental assignment. It remains strictly immutable during routine profile edits and is only altered upon the execution of an authorized **Department Transfer**, upon which a new sequential code is automatically allocated.
2. **Internal Primary Key Preservation:** For database referential integrity and audit compliance, the internal database primary key (`id BIGINT`) never changes throughout an employee's organizational lifecycle. All past historical audit records, transfer logs, and system references remain linked to this surrogate key.
3. **Sensitive Identity Data Privacy:** In strict alignment with Requirement Task B.5, **Aadhaar numbers and PAN are deliberately excluded from unrestricted global text search** to prevent inadvertent identification data leakage. Furthermore, Aadhaar digits are masked in the UI (`XXXX-XXXX-1234`) with role-restricted unmask toggles.
4. **Department Sequence Concurrency Control:** To eliminate race conditions under concurrent employee onboarding or simultaneous department transfers, sequence increments are managed using **Pessimistic Row-Level Locking** (`SELECT ... FOR UPDATE`), ensuring zero duplicate code generation.
5. **Statutory Currency & Compensation Standards:** All financial compensation figures are modeled in Indian Rupees (INR ₹) with predefined Lakhs Per Annum (LPA) filtering bands (₹1–3 LPA, ₹3–5 LPA, ₹5–10 LPA, ₹10+ LPA, and custom).

2. CANDIDATE UNDERTAKING & CERTIFICATE

CERTIFICATE OF INDEPENDENT ORIGINAL WORK

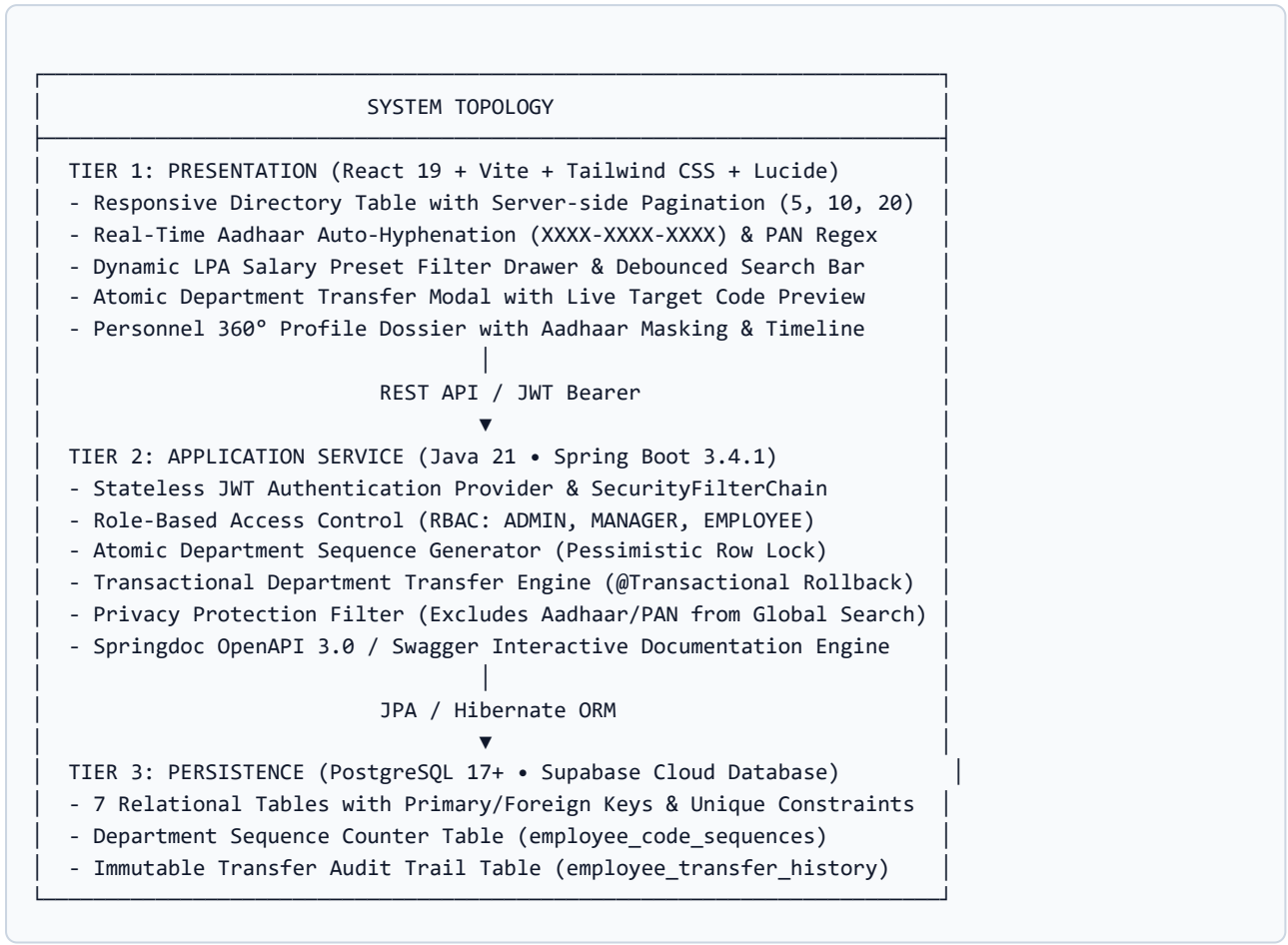
I, **Saket Rajak**, hereby certify that this technical assessment for the *Process Automation and Application Development Intern* vacancy under the **Indian Army Internship Programme (IAIP) 2026** has been independently completed by me. The solution, including all database schemas, backend RESTful APIs, Spring Security RBAC configurations, and responsive React frontend components, represents my authentic work and architectural design. All external libraries, packages, and frameworks utilized have been transparently declared in the dependency manifests below.

Candidate Signature: _____
Name: **Saket Rajak**
Email: `saketsince05@gmail.com`

Date of Submission: **31 October 2026**
Place: **Pune / Maharashtra**
Submission Deadline: **Within 48 Hours**

3. SYSTEM ARCHITECTURE & COMPONENT DESIGN

The application is architected following the standard 3-tier enterprise pattern with strict separation of concerns, stateless token authentication, and ACID transaction boundaries:



4. RELATIONAL DATABASE DEVELOPMENT (TASK C)

The database schema is implemented in PostgreSQL with strict normalization, declarative constraints, and performance indexes:

TABLE NAME	PRIMARY KEY	FOREIGN KEYS & KEY CONSTRAINTS	BUSINESS PURPOSE
`departments`	`id (SERIAL)`	`name UNIQUE, code_prefix UNIQUE`	Maintains organizational departments and assigned code prefixes (IT: I, HR: H, Fin: F, Sales: S, Admin: A).
`roles`	`id (SERIAL)`	`name UNIQUE`	System roles governing role-based access control (Admin, Manager, Employee).
`employee_code_sequences`	`department_id`	`FK -> departments(id)`	Tracks the current sequential integer per

TABLE NAME	PRIMARY KEY	FOREIGN KEYS & KEY CONSTRAINTS	BUSINESS PURPOSE
			department for atomic code generation.
`employees`	`id (BIGSERIAL)`	`employee_code UNIQUE`, `FK -> departments`, `FK -> roles`	Core employment entity storing status, joining date, and annual salary (> 0).
`employee_personal_details`	`id (BIGSERIAL)`	`employee_id UNIQUE (FK)`, `email UNIQUE`	Stores sensitive personal data: Full Name, Email, Mobile, Aadhaar, PAN, DOB, Gender, Address.
`employee_transfer_history`	`id (BIGSERIAL)`	`FK -> employees`, `FK -> departments (old/new)`	Immutable audit log capturing transfer transitions, old/new codes, dates, and justifications.
`users`	`id (BIGSERIAL)`	`username UNIQUE`, `email UNIQUE`, `FK -> employees`	Authentication credentials with BCrypt password hashes and associated RBAC role.

> **Sample Dataset Seeding:** The database has been pre-seeded with **5 Departments**, **3 Roles**, **22 diverse Personnel Records** across all four salary bands (₹1–3 LPA, ₹3–5 LPA, ₹5–10 LPA, ₹10+ LPA), historical department transfers, and default administrative users.

5. BACKEND REST API ARCHITECTURE (TASK B)

5.1 Core Endpoint Catalog

HTTP METHOD & PATH	ACCESS ROLE	KEY PARAMETERS / REQUEST BODY	DESCRIPTION
`POST /api/auth/login`	Public	`username`, `password`	Authenticates user and returns signed stateless JWT Bearer token.
`GET /api/employees`	Authenticated	`search`, `departmentId`, `roleId`, `status`, `gender`, `minSalary`, `maxSalary`, `sortBy`, `sortDir`, `pageNumber`, `pageSize`	Server-side global search, multi-facet filtering, sorting, and pagination (5, 10, 20).
`GET /api/employees/{id}`	Authenticated	`id (Path)`	Retrieves complete 360° employee dossier including personal details and transfer history.
`POST /api/employees`	Admin, Manager	`EmployeeCreateRequest (JSON)`	Registers new employee, atomically generates department sequence code, and stores personal data.
`PUT /api/employees/{id}`	Admin, Manager	`EmployeeUpdateRequest (JSON)`	Updates employee personal and employment information with duplicate checks.
`PATCH /api/employees/{id}/deactivate`	Admin	`id (Path)`	Deactivates employee account by switching status to `INACTIVE`.
`DELETE /api/employees/{id}`	Admin	`id (Path)`	Permanently deletes employee record and

HTTP METHOD & PATH	ACCESS ROLE	KEY PARAMETERS / REQUEST BODY	DESCRIPTION
			cascading personal details.
`POST /api/transfers`	Admin, Manager	`employeeId, newDepartmentId, transferDate, reason`	Executes atomic department transfer, re-allocates code, and records audit trail.
`GET /api/transfers/employee/{id}`	Authenticated	`employeeId (Path)`	Retrieves historical transfer timeline for a specific employee.
`GET /api/lookup/departments`	Public	None	Retrieves all configured departments with code prefixes.
`GET /api/lookup/preview-code/{deptId}`	Public	`deptId (Path)`	Calculates and previews the next available employee code for a department.

5.2 Technical Concurrency & Transaction Implementation

- **Atomic Code Generation Service:** Uses `@Lock(LockModeType.PESSIMISTIC_WRITE)` to acquire an exclusive lock on the departmental sequence row during increment operations. This guarantees zero duplicate codes even under simultaneous requests.
- **Department Transfer Engine:** Enclosed within `@Transactional(rollbackFor = Exception.class)`. If code generation, employee update, or audit logging encounters an exception, the entire transaction is rolled back, preventing orphaned or corrupt state.
- **Statutory Validations:** Enforced through Jakarta Bean Validation (`@Pattern` for 10-digit mobile and PAN format `ABCDE1234F`, `@DecimalMin` for positive salary, and `@Past` for Date of Birth).

6. FRONTEND WEB APPLICATION (TASK A)

The client is built using **React 19 with Vite and Tailwind CSS**, styled with a professional military/defense palette (slate, navy, and emerald accents):

6.1 Core Frontend Features

1. **Employee Management Directory Table:** Displays Employee Code, Full Name, Department, Role, Contact, Annual Salary (INR formatted), Status badge, and contextual action buttons. Supports multi-column ascending/descending sorting and server-side pagination with 5, 10, and 20 rows per page.
2. **Reactive Aadhaar Input Auto-Formatting:** Automatically injects hyphens into the Aadhaar input field as the user types (`XXXX-XXXX-XXXX`) and caps at exactly 12 numeric digits with immediate visual feedback.
3. **Statutory PAN Card Validator:** Enforces uppercase format conversion and regex validation (`[A-Z]{5}[0-9]{4}[A-Z]{1}`) in real time before submission.
4. **Combined Global Search & Multi-Filter Drawer:** Debounced global search across permitted text fields combined with simultaneous filters for Department, Role, Status, Gender, Joining Date, and **Salary LPA presets (₹1–3 LPA, ₹3–5 LPA, ₹5–10 LPA, ₹10+ LPA, or custom Min/Max)**.
5. **Department Transfer Modal:** Displays employee details, disables reassignment to current department, dynamically queries the API to display the *projected new code*, and collects an official transfer justification.
6. **Personnel 360° Profile Dossier:** Modal displaying personal, identity, and employment records with an interactive Aadhaar masking toggle (`XXXX-XXXX-1234`) and a chronological transfer audit timeline.

7. EXACT DEPENDENCY MANIFEST (RULE 8 & 218 COMPLIANCE)

In accordance with submission guidelines, only the exact dependencies required for the project runtime are installed:

LAYER	GROUP ID / PACKAGE	ARTIFACT ID / MODULE	EXACT VERSION	RUNTIME FUNCTION
Backend	`org.springframework.boot`	`spring-boot-starter-web`	`3.4.1`	REST API dispatching, Jackson JSON serialization
Backend	`org.springframework.boot`	`spring-boot-starter-data-jpa`	`3.4.1`	Spring Data JPA, Hibernate ORM, Transaction Manager
Backend	`org.springframework.boot`	`spring-boot-starter-validation`	`3.4.1`	Jakarta Bean Validation 3.0 annotations
Backend	`org.springframework.boot`	`spring-boot-starter-security`	`3.4.1`	SecurityFilterChain, BCryptPasswordEncoder, RBAC
Backend	`org.postgresql`	`postgresql`	`42.7.4`	PostgreSQL Production JDBC Database Driver
Backend	`org.springdoc`	`springdoc-openapi-`	`2.8.3`	OpenAPI 3.0 & Swagger UI (`/swagger-ui.html`)

LAYER	GROUP ID / PACKAGE	ARTIFACT ID / MODULE	EXACT VERSION	RUNTIME FUNCTION
		`starter-webmvc-ui`		
Backend	`io.jsonwebtoken`	`jjwt-api / impl / jackson`	`0.12.6`	Stateless HS256 JWT creation, signing & parsing
Frontend	`react` / `react-dom`	UI Library	`19.3.1`	Component lifecycle and reactive state rendering
Frontend	`vite`	Build Tool & Dev Server	`8.3.5`	Fast ES-module development and production bundling
Frontend	`tailwindcss`	CSS Framework	`3.4.17`	Responsive enterprise styling and utility classes
Frontend	`axios`	HTTP Client	`1.8.2`	API communication with Bearer token interceptor
Frontend	`lucide-react`	Icon Set	`1.16.0`	Interface iconography

8. INSTALLATION & VERIFICATION INSTRUCTIONS

8.1 Pre-Seeded Default Authentication Credentials

USER ROLE	USERNAME	PASSWORD	AUTHORIZATION SCOPE
Admin	`admin`	`Admin@123`	Full CRUD, Department Transfer, Account Deactivation, System Oversight
Manager	`manager`	`Manager@123`	Employee Directory View, Edit Details, Initiate Department Transfer
Employee	`employee`	`Employee@123`	Read-Only Directory View & Personal Dossier Review

8.2 Restoration & Local Execution Commands

```
# 1. Database Execution
Execute schema.sql and seed.sql in PostgreSQL or Supabase SQL Editor:
psql -U postgres -d postgres -f database/schema.sql
psql -U postgres -d postgres -f database/seed.sql

# 2. Backend Execution (Spring Boot REST API)
cd backend
mvn clean compile
mvn spring-boot:run
# Live Swagger Documentation: http://localhost:8080/swagger-ui.html

# 3. Frontend Execution (React + Vite Dashboard)
cd frontend
npm install
npm run dev
# Live Client Dashboard: http://localhost:5173
```

9. SUBMISSION DELIVERABLES SUMMARY

- **GitHub Repository URL:** https://github.com/Saketrajak/Nxt-Gen-Employee-Management-System
- **Source Code Bundle:** `IAIP_EMS_Saket_Rajak.zip` (Sanitized, excluding `node_modules`, `target`, and passwords as required by Rule 220).
- **OpenAPI 3.0 Documentation:** `docs/openapi_spec.json` and interactive Swagger UI at `/swagger-ui.html`.
- **Postman Test Collection:** `docs/IAIP_EMS_Postman_Collection.json` containing pre-configured requests for all endpoints.
- **Submission PDF Report:** This document, incorporating all design specifications, assumptions, and candidate undertaking.